

Normalização de Base de Dados — Sistema de Gestão de Funcionários

Universidade Licungo — Faculdade de Ciências e
Tecnologias

Curso de Licenciatura em Informática · Trabalho II

Objetivo

Este repositório documenta o processo de normalização de uma base de dados de funcionários, partindo de uma única tabela não normalizada (0FN) — mantida originalmente numa folha de cálculo — até um esquema relacional

bem desenhado, aplicando sucessivamente a 1.^a, 2.^a, 3.^a e 4.^a Formas Normais (1FN → 4FN).

O ponto de partida foi o ficheiro

`Dados_Nao_Normalizados_Funcionarios.xlsx`, com 16 registos de funcionários, contendo dados pessoais, morada, dados profissionais (cargo/função), filhos (até 3, em colunas repetidas) e contactos telefónicos (até 3, em colunas repetidas).

Estrutura do repositório

```

.
├── README.md                    # este
ficheiro
├── /documentos
│   └── Trabalho2_Normalizacao.pdf #
análise e justificação de cada forma normal (1FN
a 4FN),
│                                   # com as
tabelas resultantes de cada fase e as
cardinalidades
├── /diagramas
│   └── MER_Funcionarios.png      # Modelo
Entidade-Relacionamento final, com
cardinalidades
└── /sql
    ├── ddl_schema.sql            # CREATE
TABLE de todas as entidades, com PKs e FKs
    └── queries_exemplo.sql        #
queries de exemplo (JOIN) que reconstituem os
dados originais

```

Como consultar os artefactos

1. Análise da normalização (1FN → 4FN): abrir

`/documentos/Trabalho2_Normalizacao.pdf`. O

documento segue

esta ordem: identificação dos problemas na tabela

original → 1FN → 2FN → 3FN → 4FN → cardinalidades

→

diagrama MER → síntese do esquema final. Cada fase

inclui as tabelas resultantes e a justificação das

dependências eliminadas.

2. Diagrama ER: abrir

`/diagramas/MER_Funcionarios.png` para visualizar as entidades finais (FUNCIONARIO, CARGO, FUNCAO, POSTO_TRABALHO, CIDADE, PROVINCIA, PAIS, FILHO, TELEFONE), os seus atributos, chaves primárias/estrangeiras e as cardinalidades de cada relacionamento (todas 1:N).

3. Scripts SQL: em `/sql`, o ficheiro `ddl_schema.sql` cria todas as tabelas do esquema normalizado; o ficheiro `queries_exemplo.sql` contém consultas com `JOIN` que reconstituem a informação tal como aparecia na tabela original (0FN), demonstrando que nenhuma informação foi perdida no processo de normalização.

Exemplos de consulta com JOIN

O ficheiro `/sql/queries_exemplo.sql` contém as queries completas. Seguem dois exemplos que mostram como a informação original (0FN) é reconstituída a partir do esquema normalizado:

1. Ficha completa do funcionário — junta `FUNCIONARIO` com `CIDADE`, `PROVINCIA`, `PAIS`, `CARGO`, `FUNCAO` e `POSTO_TRABALHO`, devolvendo a morada, o cargo e a função já "traduzidos" dos códigos para texto legível:

```
SELECT f.bi, f.nome, f.email,  
       CONCAT(f.rua, ' ', f.n.º, ' ', f.numero, ' ', f.f.bairro) AS endereco,  
       ci.nome AS cidade, pv.nome AS provincia,  
       pa.nome AS pais,
```

```

        ca.designacao AS cargo, fu.designacao AS
funcao,
        pt.designacao AS posto_trabalho,
f.data_admissao
FROM FUNCIONARIO f
JOIN CIDADE      ci ON ci.cod_cidade    =
f.cod_cidade
JOIN PROVINCIA   pv ON pv.cod_provincia =
ci.cod_provincia
JOIN PAIS        pa ON pa.cod_pais      =
pv.cod_pais
JOIN CARGO       ca ON ca.cod_cargo     =
f.cod_cargo
JOIN FUNCAO      fu ON fu.cod_funcao    =
f.cod_funcao
JOIN POSTO_TRABALHO pt ON pt.cod_posto  =
f.cod_posto
ORDER BY f.nome;

```

2. Filhos e telefones agregados — junta `FUNCIONARIO` com `FILHO` (e, em separado, com `TELEFONE`), usando `LEFT JOIN` + `GROUP_CONCAT`, para reconstituir o equivalente às antigas colunas "Filho 1/2/3" e "Celular 1/2/3", sem o limite fixo de 3 posições:

```

SELECT f.bi, f.nome,
       COUNT(fi.id_filho) AS num_filhos,
       GROUP_CONCAT(fi.nome_filho SEPARATOR ' |
') AS filhos
FROM FUNCIONARIO f
LEFT JOIN FILHO fi ON fi.bi_funcionario = f.bi
GROUP BY f.bi, f.nome
ORDER BY f.nome;

```

Nota: `GROUP_CONCAT` é sintaxe MySQL; em PostgreSQL o equivalente é `STRING_AGG(coluna, ' | ')`.

O ficheiro `queries_exemplo.sql` inclui ainda uma 4.^a query que combina tudo numa reconstituição linha a linha igual à tabela original, e uma 5.^a query analítica (contagem de funcionários por cargo e província) para ilustrar uma vantagem prática de ter os dados normalizados.

Resumo do esquema final

Entidade	Descrição
FUNCIONARIO	Dados pessoais e profissionais de cada funcionário (BI como PK)
CARGO	Tabela de referência de cargos (elimina dependência parcial)
FUNCAO	Tabela de referência de funções (elimina dependência parcial)
POSTO_TRABALHO	Locais/delegações de trabalho
CIDADE	Cidades, ligadas a uma província
PROVINCIA	Províncias, ligadas a um país (elimina dependência transitiva)
PAIS	País
FILHO	Filhos de cada funcionário (elimina grupo repetitivo / 1FN e 4FN)

Entidade	Descrição
TELEFONE	Contactos telefónicos de cada funcionário (idem)

Todos os relacionamentos identificados têm cardinalidade **1:N**; não existem relacionamentos N:M neste modelo.

Ferramentas utilizadas

- Análise e documento: Python (extração/estruturação dos dados) + ReportLab (geração do PDF)
- Diagrama ER: Graphviz
- Base de dados: SQL padrão (compatível com MySQL/PostgreSQL)

Autor

Trabalho realizado no âmbito da disciplina de Bases de Dados, Curso de Licenciatura em Informática, Universidade Licungo.